



Projet BTS -Gestion des déchets **en restauration scolaire**

Nom de l'étudiant : ABALLEA Killian | G Ronan | D Younn - Spécialité : Cybersécurité Informatique
Électronique et Réseaux, option Informatique et Réseaux

P2026 :

Aballea Killian

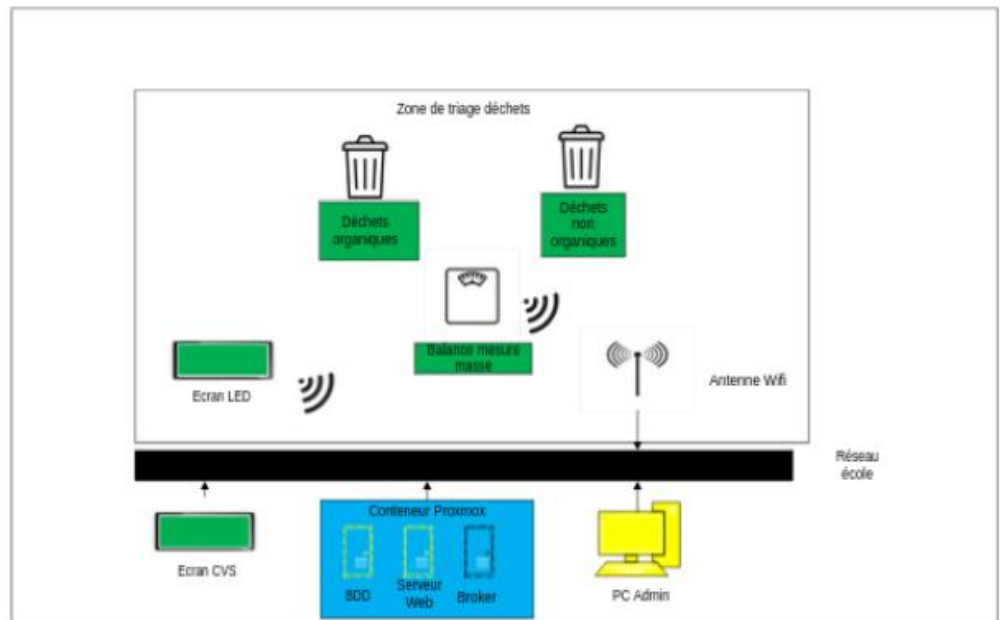
Spécialité Informatique et Réseaux

Dossier technique du projet - partie individuelle

1. Situation dans le projet	3
1.1 Synoptique de la réalisation	2
1.2 Schéma réseau	4
1.3 Diagramme de déploiement	5
1.4 Diagramme des exigences	6
1.5 Sauvegarde du Projet	7
2. Description de la partie personnelle	7
2.1 Diagramme de cas d'utilisation :	8
2.2 Objectif :	9
2.3 Grandes phases de ma partie du Projet :	10
2.4 Réalisation de la fonction et/ou cas d'utilisation	11
2.5 Conception détaillée	15
2.5.1 Développement logiciel	15
2.5.1.1 Tâche 2 : Configuration de la BDD	15
2.5.1.2 Tâche 3 : Configuration du broker MQTT	16
2.5.2 Configuration/Installation réseau ou de services	21
2.5.2.1 Tache 1 : Installation sur le conteneur Proxmox des services	21
2.5.2.1 Tache 4 : Réception des données sur le broker et envoie vers la BDD	22
2.6 Tests unitaires	26
2.7 Problèmes rencontrés	27
3. Bilan de la réalisation personnelle	27
3.1 Points du projet validés	27
3.2 Parties à développer	27
3.3 Améliorations possibles	27
3.4 Conclusion personnelle	28

1.Situation dans le projet

1.1Synoptique de la réalisation



Le synoptique ci-dessous présente l'architecture générale du système de gestion des déchets mis en place dans le cadre du projet.

Dans le cadre de ce projet, ma mission a consisté à déployer l'ensemble de l'infrastructure serveur indispensable au fonctionnement du système. J'ai notamment installé et configuré le **serveur web**, permettant à Ronan de développer l'interface utilisateur, ainsi que le **broker MQTT** et la **base de données MariaDB**, qui assurent la réception, la sécurisation et le stockage des données envoyées par les balances connectées.

Description du fonctionnement du système :

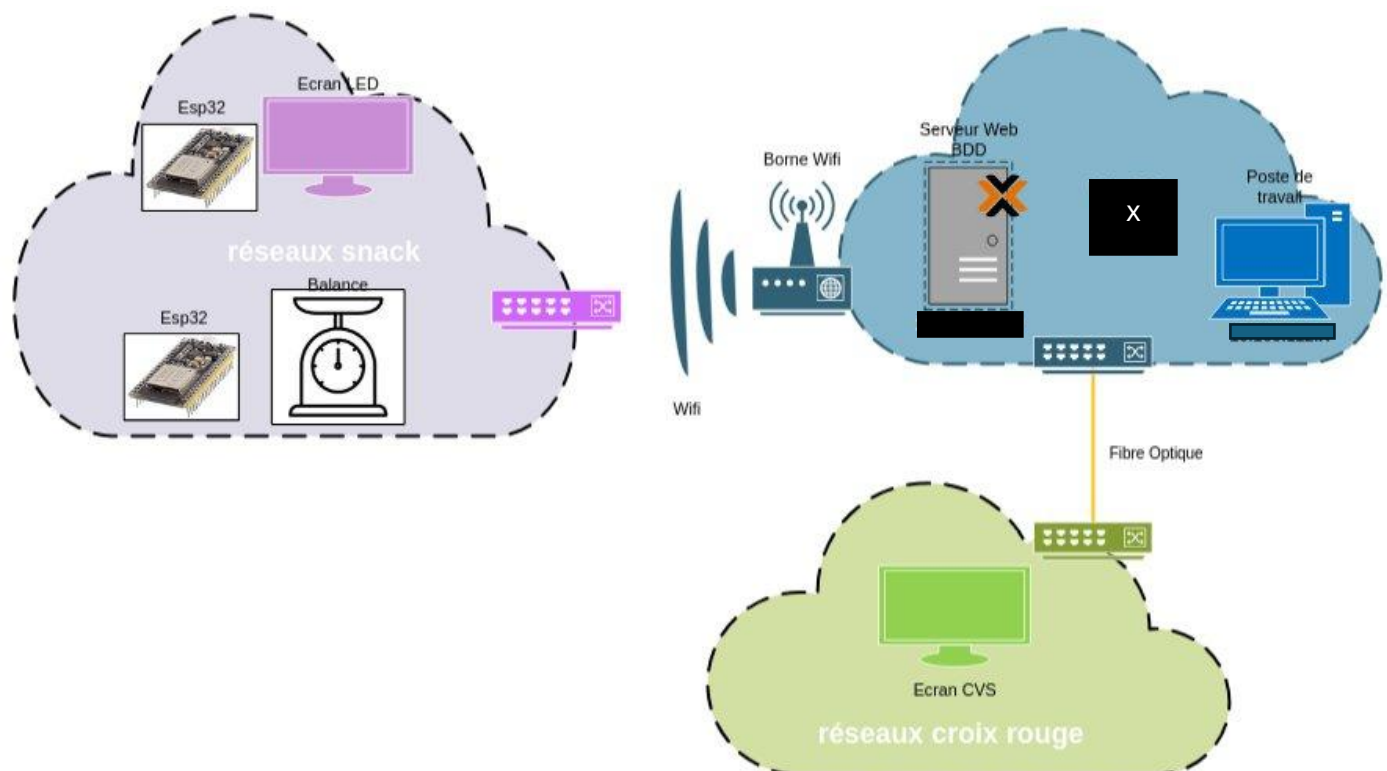
Le système de gestion des déchets mis en place repose sur une architecture IoT complète permettant de collecter, transmettre, traiter et afficher les données issues des balances connectées. Le fonctionnement global s'articule autour de quatre modules principaux : la balance connectée, le broker MQTT, le script Python de traitement et la base de données MariaDB, auxquels s'ajoute l'interface web développée par l'un des membres de l'équipe.

Lorsqu'un déchet est pesé, la balance connectée mesure la masse et génère un message contenant plusieurs informations : le poids, le type de déchet, la date et l'identifiant de la balance. Ces données sont publiées automatiquement sur un topic MQTT défini pour chaque balance. Le broker Mosquitto, que j'ai installé et sécurisé, reçoit ces messages et les met à disposition des clients autorisés.

Un script Python, que j'ai développé, est abonné aux topics des balances. À chaque message reçu, il vérifie l'intégrité du JSON, contrôle la validité des champs, normalise les données puis les insère dans la base MariaDB. Cette base, que j'ai conçue et configurée, stocke l'ensemble des pesées de manière structurée, en respectant les contraintes définies dans le MCD et le schéma relationnel.

Enfin, le serveur web Node.js permet au personnel de restauration de consulter les données via une interface graphique. Ce module, développé par Ronan, interroge la base de données pour afficher les pesées, les menus associés et les statistiques nécessaires au suivi des déchets.

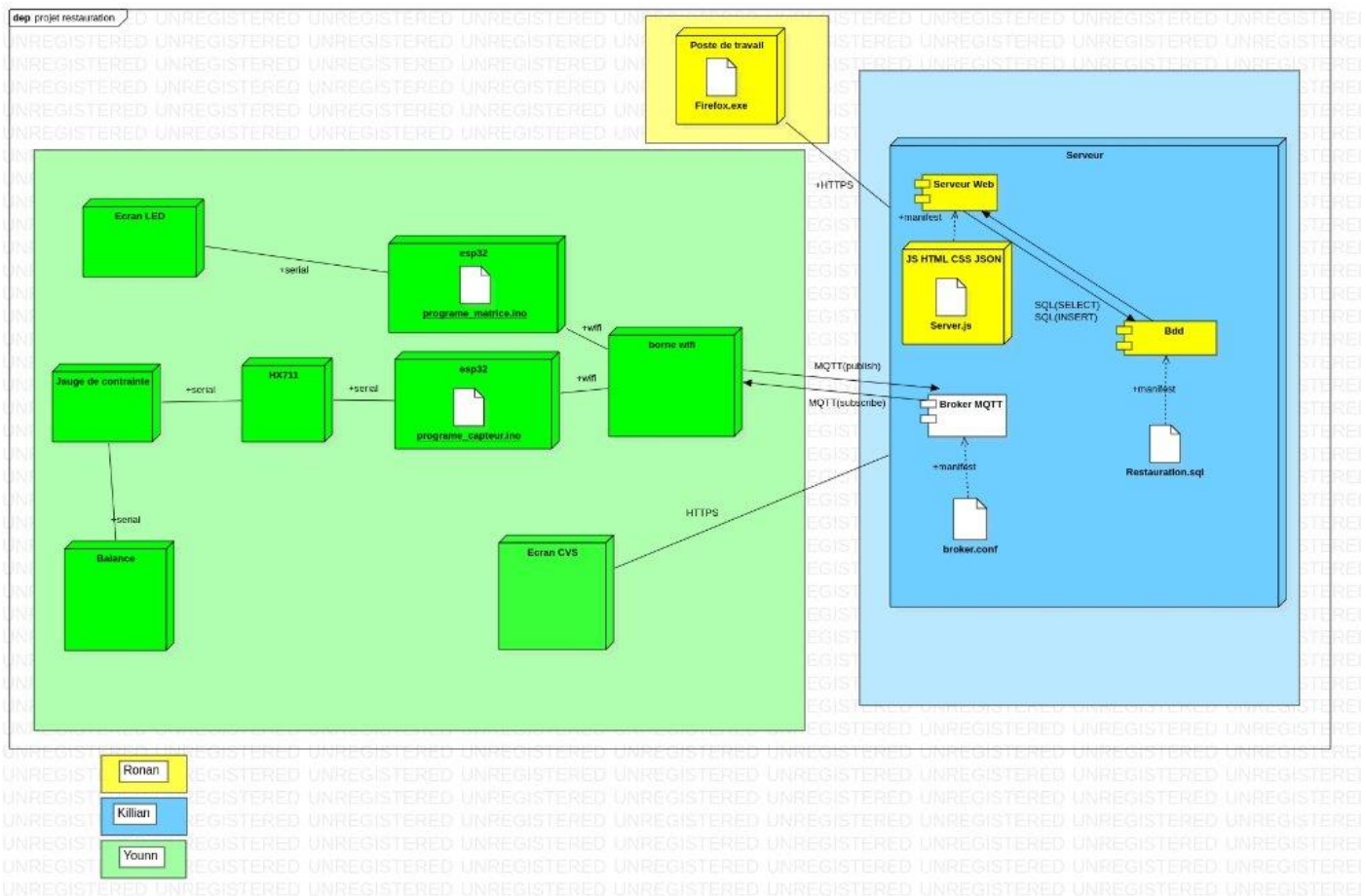
1.2 Schéma réseau



Le schéma représente l'architecture réseau complète du projet en mode production, organisée en trois zones distinctes :

- **Réseau Snack** : Cette zone regroupe les équipements IoT du système, notamment deux **ESP32**. L'un est relié à la **balance**, l'autre à l'**écran LED**. Ces appareils communiquent en **Wi-Fi** via une borne Wifi.
- **VLAN X** : C'est le cœur du système. On y trouve le **serveur Web/BDD** qui héberge le site, le broker MQTT et la base de données. Un **poste de travail** permet l'administration et la consultation. La **borne Wi-Fi** assure la liaison entre les ESP32 et le serveur.
- **Réseau Croix Rouge** : Cette zone contient l'**écran CVS**, connecté via **fibre optique** au VLAN 160. Il récupère les informations envoyées par le serveur pour l'affichage.

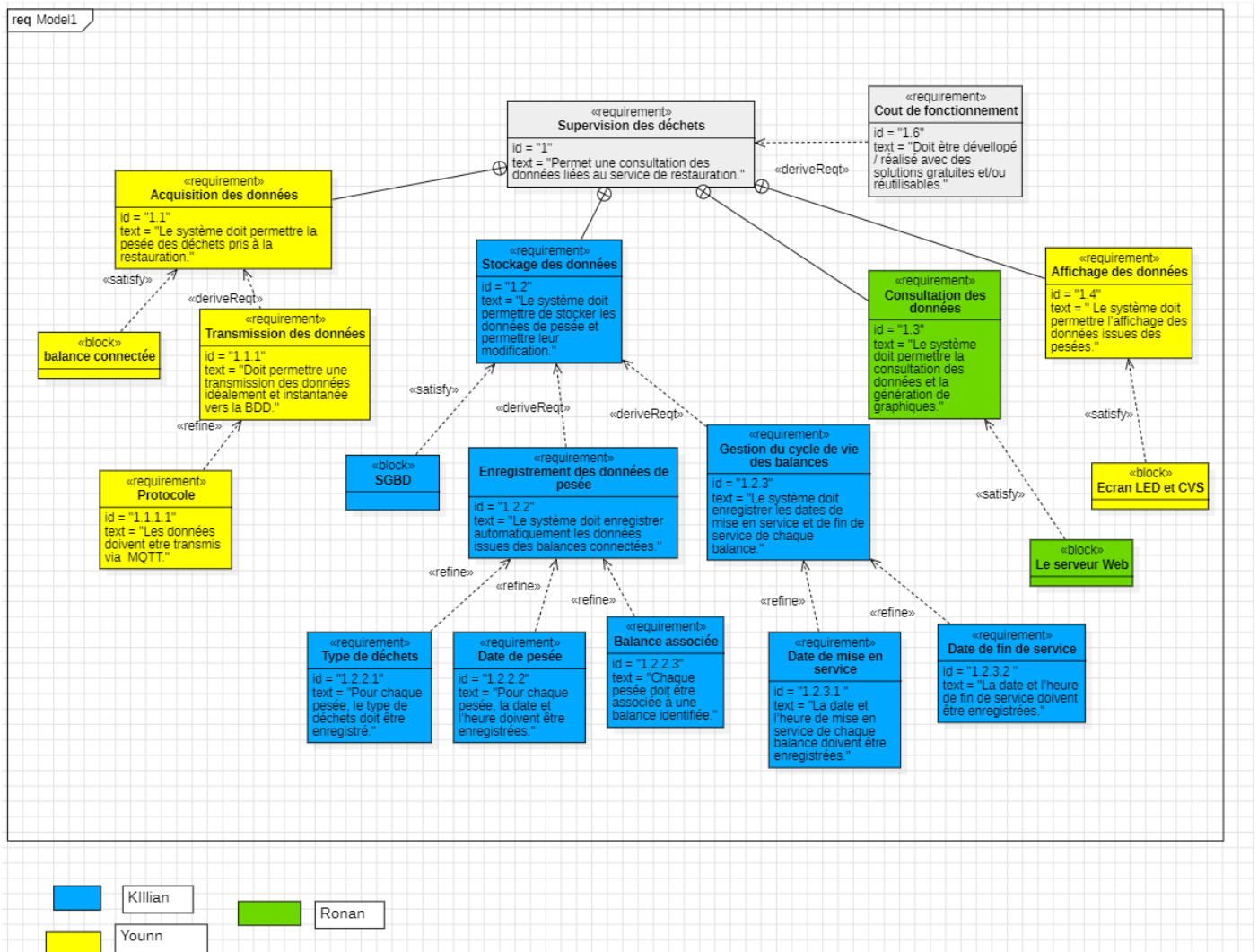
1.3 Diagramme de déploiement



Le diagramme de déploiement ci-dessus représente l'architecture complète du système. Ma partie (en bleu) comprend les nœuds suivants :

- **Broker MQTT (Mosquitto)** : reçoit les messages publiés par les balances via le protocole MQTT et les transmet au script Python.
- **Base de données MariaDB (BDD)** : stocke de manière structurée toutes les pesées reçues, les menus et les utilisateurs.
- **Serveur Web Node.js** : constitue le point d'entrée unique pour interroger la BDD, il assure également la sécurité des échanges via HTTPS.

1.4 Diagramme des exigences



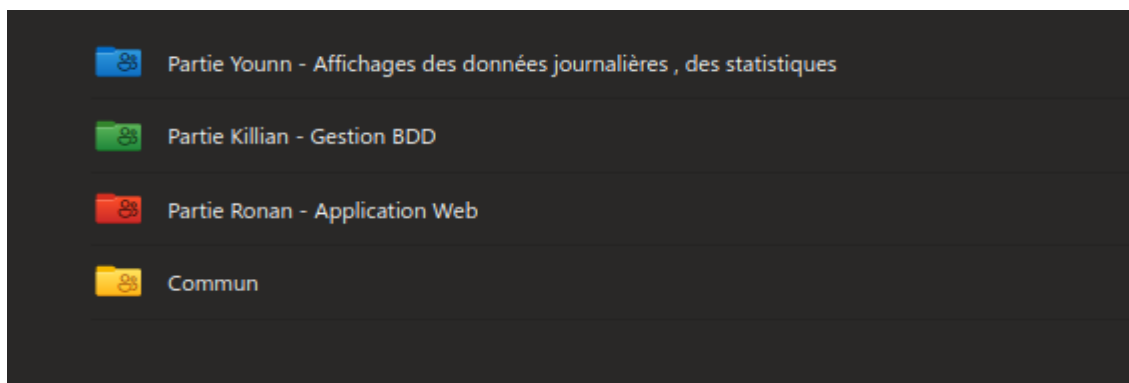
reçoit les messages MQTT, vérifie leur intégrité, extrait les champs obligatoires et insère automatiquement chaque pesée dans la table `DonneeCollecte`.

Enfin, l'exigence **1.2.3 – Gestion du cycle de vie des balances** fait également partie de ma mission. Elle impose de suivre l'état de chaque balance (mise en service, fin de service), ce qui a été intégré dans la structure de la base de données et dans la logique du script de traitement.

1.5 Sauvegarde du Projet

1er moyen de sauvegarde utilisé : OneDrive

Capture d'écran du dossier OneDrive :



Chaque membre du projet dispose de son propre espace OneDrive personnel, auquel s'ajoute un OneDrive partagé destiné au stockage et à l'échange des documents communs.

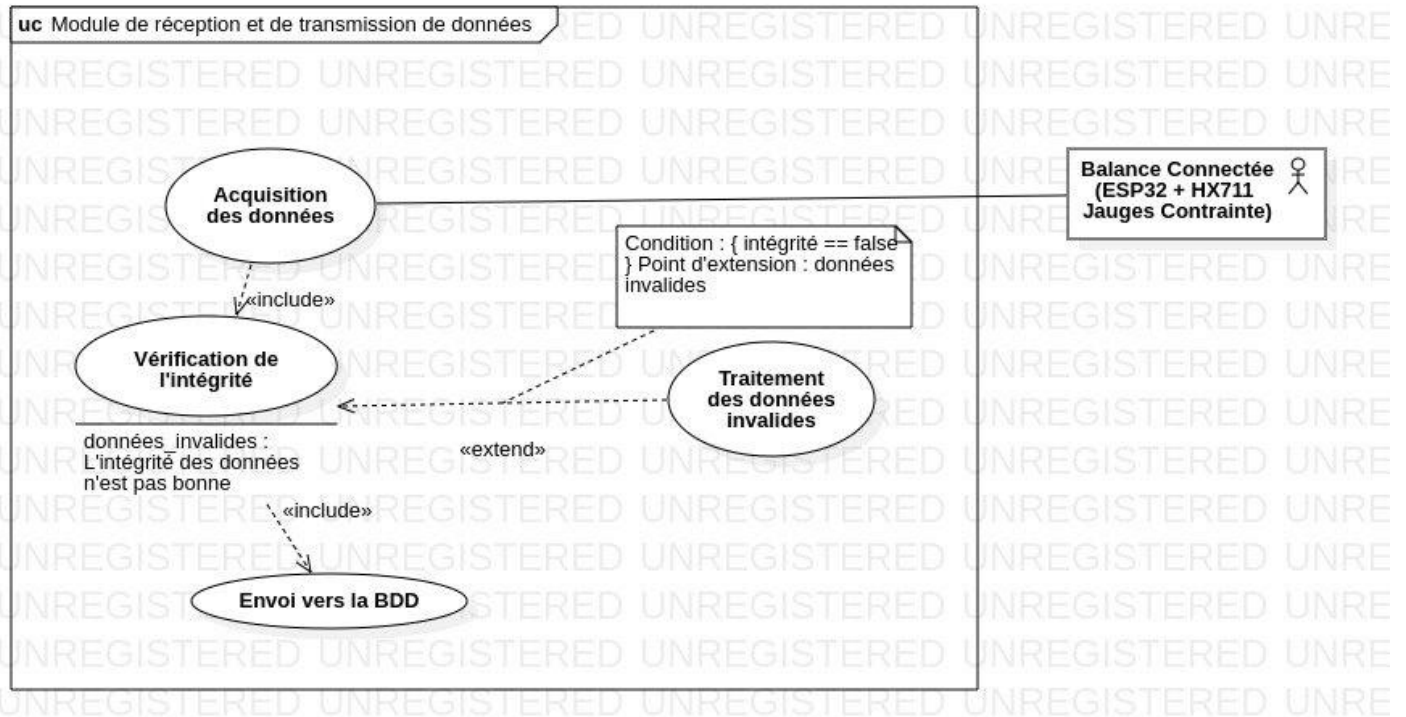
2. Description de la partie personnelle

Dans le cadre de ce projet, j'ai eu pour mission de mettre en place l'ensemble de l'infrastructure technique permettant la collecte, la transmission et le stockage des données issues des balances connectées. Mon rôle s'est articulé autour de trois axes principaux : l'installation du serveur web, la configuration du broker MQTT et la conception de la base de données MariaDB, afin d'assurer un système fiable, sécurisé et opérationnel.

Ainsi, le module développé par **Ronan G** repose directement sur les données que je collecte et structure. Il a pu concevoir l'interface web et les outils de visualisation grâce à l'API et à la base de données que j'ai mises en place. J'intervenais en second dans la chaîne technique, entre la balance connectée et l'interface web : je recevais les données brutes, je les validais, je les sécurisais et je les rendais exploitables pour l'application.

Pour finir, **Younn D** était quant à lui responsable de la partie électronique et du fonctionnement de la balance connectée. Son travail constituait le point de départ du système, puisqu'il assurait la mesure physique des déchets et l'envoi des données au broker MQTT que j'avais configuré.

2.1 Diagramme de cas d'utilisation



Le module de réception et de transmission de données est constitué de :

- Un **broker MQTT**
- Un **serveur de base de données**
- Une **base de données nommée Restauration**

Ils sont tous hébergés sur le Proxmox.

La balance connectée publie ses données vers le Broker MQTT.

Le script vérifie l'intégrité des données, les traite et les envoie vers la BDD.

Scénario optimal :

- Le script se Sub au Broker MQTT.
- La Balance / Programme Tests fait un Pub vers le Broker MQTT.
- Le script récupère les données envoyées sur le Broker MQTT.
- Le script vérifie l'intégrité des données, les traite et les envoie vers la BDD.

Une fois les données reçues, le script procède au décodage du message JSON, contrôle la présence et la cohérence des champs obligatoires, puis distingue les données valides des données invalides. Les données incorrectes déclenchent le cas d'utilisation *Traitement des données invalides*, tandis que les données valides sont normalisées, converties si nécessaire et enrichies d'un horodatage serveur. Le script exécute ensuite une requête SQL d'insertion dans la base Restauration et valide l'écriture via un commit, rendant les informations immédiatement disponibles pour l'interface web et les écrans d'affichage.

2.2 Objectif :

Collecte et stockage des données :

- Collecte des données issues des capteurs et services prévus via les protocoles prévus (MQTT, API).
- Stockage des données de façon manière fiable et structurée dans une BDD relationnelle.
- Garantir l'intégrité, la disponibilité et la cohérence des données transmises.

Conception de la BDD :

- Définir le **MCD**, le **schéma relationnel** ainsi que les **contraintes**.
- Générer les requêtes d'interrogations SQL afin d'assurer le bon fonctionnement du système (sélection, insertion, filtrage).

Mise en place de l'infrastructure serveur :

- *Installer et configurer le Conteneur Proxmox avec les services adéquats :*
 - Serveur **WEB**
 - Serveur **BDD : MariaDB**
 - Services supplémentaires
 - Déployer les services et vérifier leur bon fonctionnement en conditions réelles.

Sécurité et conformité :

- **Vérifier la conformité du système au RGPD :**
 - Minimisation des données collectées.
 - Limitation des accès aux seules personnes autorisées.
 - Conservation des données dans un environnement sécurisé.

Sécuriser les accès aux services :

- Authentification et autorisation.
- Chiffrement des communications (HTTPS, certificats)
- Gestion des permissions sur la BDD

Définir une liste de tests d'intrusion essentiels auxquels le système doit résister :

- Injection SQL
- Brute force sur les accès
- Escalade de privilèges
- Attaques sur le broker MQTT (DDoS, spoofing, topic hijacking [**vol de topic**])
- Analyse des ports et services exposés via **Wireshark**
- Vérification des failles de configuration (CORS, headers, certificats)

Gestion du broker :

- Installer et configurer le broker MQTT.
- Définir les topics
- Assurer la réception, la validation et la gestion automatique des messages dans la base de données.

2.3 Grandes phases de ma partie du Projet :

Phase	Description	Acteurs
1. Analyse du besoin	Compréhension du fonctionnement des balances, du besoin du service restauration, des contraintes techniques.	Équipe projet, Responsable restauration
2. Conception de l'architecture	Définition de l'architecture : Balance → MQTT → Python → MariaDB → Interface Web.	Équipe projet
3. Mise en place du broker MQTT	Installation, configuration, création des utilisateurs, tests de publication/souscription	Admin Système (Moi)
4. Conception et création de la base de données	Modélisation, création des tables, contraintes, index, tests d'insertion.	Admin BDD (Moi)
5. Développement du programme Python	Réception MQTT, validation JSON, traitement, insertion SQL, gestion des erreurs.	Admin Système (Moi)

Diagramme de GANTT Prévisionnel :

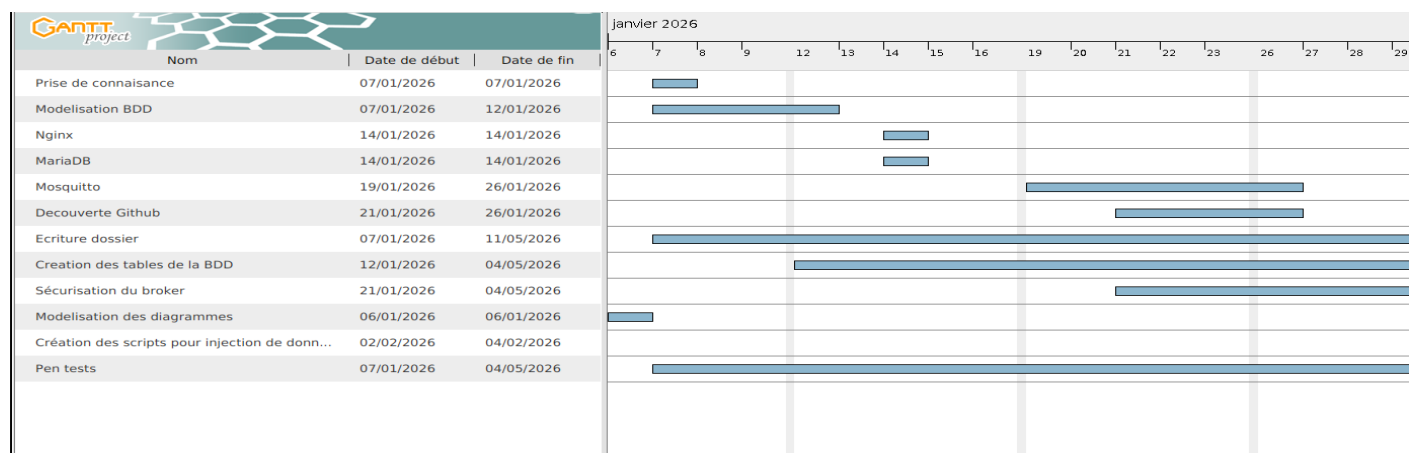
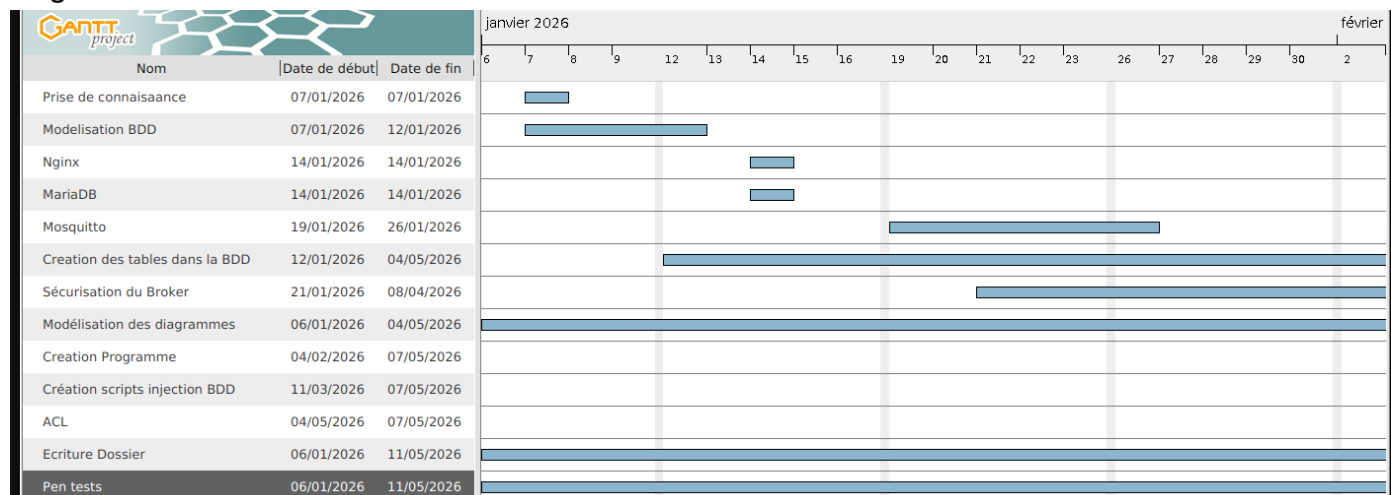


Diagramme de GANTT Définitive :



Extrait du carnet de bord :

3/30/2026	Dernier essai sécurisation des services	Certificat non fonctionnel avec Ronan	Désactiver tous les moyens de sécurisation pour RP2
3/31/2026	Finalisation du diagramme d'existence + Réflexion diagramme cas d'utilisation	UC peut être non adaptée à ma partie	Réflexion sur mon UC
4/1/2026	Revu de projet N°2 + Avancement dossier commun partie description détaillée	x	x
4/2/2026			
4/3/2026			BTS BLANC
4/4/2026			
4/5/2026			
4/6/2026	x	x	x
4/7/2026	Avancement Dossier Commun et Individuel	x	x
4/8/2026	Implémentation TLS sur le Broker Mosquitto, Tests Balance et Programme OK ; Ajout des accès lists pour les topics ainsi que réflexion sur les topics et sur contraintes à mettre sur les tables : par exemple ENUM	Problème nom du CN qui doit être l'adresse IP du serveur et problème de lecture sur le CA.cert car l'utilisateur balance_test par dans le groupe mosquito	x
4/9/2026			
4/10/2026			

Les diagrammes de GANTT ci-dessous présentent la planification du projet et son évolution entre la phase initiale et la réalisation finale. Le **diagramme prévisionnel** illustre la répartition des tâches telles qu'elles avaient été planifiées au début du projet, tandis que le **diagramme définitif** montre les ajustements effectués au fil du développement.

Dans la version prévisionnelle, les tâches principales étaient réparties de manière linéaire : modélisation de la base de données, installation des services (Nginx, MariaDB, Mosquitto), puis développement des scripts et rédaction du dossier. Cette organisation visait à établir une progression logique, du déploiement technique vers la validation fonctionnelle.

Au cours du projet, plusieurs ajustements ont été nécessaires. Le **diagramme définitif** met en évidence les écarts suivants : certaines tâches ont été prolongées, notamment la **sécurisation du broker MQTT** et la **création des scripts d'injection de données**, qui ont demandé davantage de tests et de corrections. De nouvelles activités ont également été ajoutées, comme la **mise en place des ACL** et les **tests de pénétration (Pen-tests)**, afin de renforcer la sécurité du système. Ces modifications montrent une évolution naturelle du projet : l'évolution technique du projet et la volonté d'obtenir un système complet, fiable et conforme aux exigences du cahier des charges.

2.4 Réalisation de la fonction et/ou cas d'utilisation

Dans cette partie, je vais présenter les quatre activités principales que j'ai réalisées au cours du projet. Elles correspondent aux différentes étapes techniques nécessaires pour assurer la collecte, la transmission, le traitement et le stockage des données issues des balances connectées. Chaque activité s'inscrit dans la continuité de la précédente et contribue au fonctionnement global du système.

Tâches réalisées

Tâche 1 : Installation du conteneur Proxmox et des services

(Installation de services)

Tâche 2 : Configuration de la BDD

(Développement logiciel)

Tâche 3 : Configuration du Broker MQTT

(Configuration de services)

Tâche 4 : Réception des données sur le broker et envoi vers la BDD (Développement logiciel)

Tâche 1 : Installation du broker et des services

Présentation de la tâche :

Cette tâche consiste à installer et configurer l'ensemble des services nécessaires au fonctionnement du système de pesée automatisé. Le conteneur Proxmox héberge trois composants essentiels :

- Un serveur web Node.js
- Une base de données MariaDB
- Un broker MQTT Mosquitto

Ces services constituent l'infrastructure centrale permettant de recevoir les données des balances, de les stocker et de les rendre accessibles à l'application web.

Justifications des choix :

Node.js (serveur web)

Node.js a été choisi car :

- Il offre une **excellente rapidité d'accès** aux données grâce à son modèle asynchrone,
- Il constitue le **point d'entrée unique** pour interroger la base de données,
- Il permet une **traçabilité avancée** via la génération de logs,
- Il facilite la mise en place d'un **chiffrement SSL/TLS**, indispensable pour sécuriser les échanges.

MariaDB (base de données)

MariaDB a été choisie car :

- Elle est **libre de droit**, ce qui correspond aux contraintes pédagogiques,
- Elle est **compatible MySQL**, facilitant l'administration et les outils,
- Elle offre une **excellente stabilité et rapidité**,
- Elle dispose d'outils de logs permettant une **traçabilité fine des accès**.

Mosquitto (broker MQTT)

Mosquitto a été choisi car :

- Il implémente nativement le protocole **MQTT**, standard dans l'IoT,
- Il permet une **transmission proche du temps réel**, essentielle pour les pesées,
- Il consomme très peu de ressources, idéal pour un conteneur,
- Il supporte **SSL/TLS** et la gestion d'identifiants pour chaque balance.

Tache 2 : Configuration de la BDD

Présentation de la tâche :

Cette tâche consiste à concevoir la structure de la base de données (MCD/MLD), créer les tables nécessaires et configurer les utilisateurs. La base doit permettre de stocker les pesées journalières, les types de déchets, les menus associés et les comptes utilisateurs.

Justification du choix technologique :

Le choix de MariaDB est cohérent avec les besoins du projet :

- Stockage structuré des données (poids, dates, types de déchets, menus),
- Gestion des relations entre les tables,
- Performances adaptées à des insertions fréquentes,
- Sécurité renforcée grâce aux utilisateurs dédiés et aux logs d'accès.

Tâche 3 — Configuration du broker MQTT

Présentation succincte de la tâche :

Dans cette partie du projet, j'ai été responsable de l'installation, de la configuration et de la sécurisation du broker MQTT. Ce composant joue un rôle central dans notre architecture, car il assure la transmission des données entre les balances connectées et le script Python chargé de les traiter. La mise en place du broker a nécessité plusieurs étapes : définition des topics, création des utilisateurs, configuration des ACL, activation du chiffrement TLS et réalisation de tests de communication.

Sécurisation du Broker :

La sécurisation du broker a été un point essentiel de mon travail. J'ai mis en place plusieurs mécanismes complémentaires :

- **Authentification par utilisateur** : chaque balance possède son propre identifiant MQTT, ce qui permet une traçabilité précise des données envoyées.
- **ACL (Access Control List)** : elles définissent précisément quels topics chaque utilisateur peut lire ou écrire, empêchant ainsi toute publication non autorisée.
- **Chiffrement TLS** : j'ai généré une autorité de certification interne (CA), un certificat serveur et une clé privée afin de chiffrer toutes les communications entre les balances, le script Python et le broker.
- **Désactivation des connexions anonymes** : seules les entités authentifiées peuvent se connecter au broker.

L'ensemble de ces éléments sera détaillé plus en profondeur dans la section dédiée à la configuration TLS et aux certificats.

Définition des topics :

Les topics ont été définis en équipe afin de garantir une structure claire, évolutive et adaptée aux besoins de chacun. Avec **Younn**, qui s'occupait de la balance connectée, nous avons défini la structure suivante :

<pre>balance/<nom_utilisateur>/data -> Topic principal balance/<nom_utilisateur>/data-2 -> Topic de secours</pre>

Travail d'équipe :

La configuration du broker a été réalisée en coordination avec les autres membres du groupe :

- Avec **Ronan**, j'ai défini les topics et les identifiants nécessaires pour que la balance puisse publier correctement ses données.
- Avec **Younn**, j'ai structuré la base de données et les flux de données afin que son interface web puisse récupérer les informations de manière fiable et sécurisée.

Cette collaboration nous a permis d'assurer une cohérence complète entre nos 3 parties distinctes.

Justification du choix technologique

Mosquitto est parfaitement adapté car :

- Il est **léger**,
- Il est **fiable**,
- Il est **simple à configurer**,
- Il permet une **authentification par utilisateur**,
- Il supporte **TLS**, indispensable pour sécuriser les données envoyées par les balances.

Tâche 4 — Collecte et stockage des données des balances

Présentation succincte de la tâche

Cette tâche consiste à développer un programme Python chargé d'assurer la réception, le traitement et l'insertion automatique des données envoyées par les balances connectées. Pour cela, le script se connecte au broker MQTT, s'abonne aux topics définis pour chaque balance, puis récupère en temps réel les messages publiés.

À chaque réception, les données sont vérifiées, décodées et normalisées afin de garantir leur intégrité. Une fois validées, elles sont automatiquement insérées dans la base de données MariaDB, où elles pourront être consultées et exploitées par l'interface web.

Un second programme a également été développé afin de générer des données de test. Celui-ci permet de simuler le comportement d'une balance et de valider l'ensemble de la chaîne de traitement, du broker MQTT jusqu'à la base de données.

Justification du choix technologique

Python a été retenu pour cette tâche car :

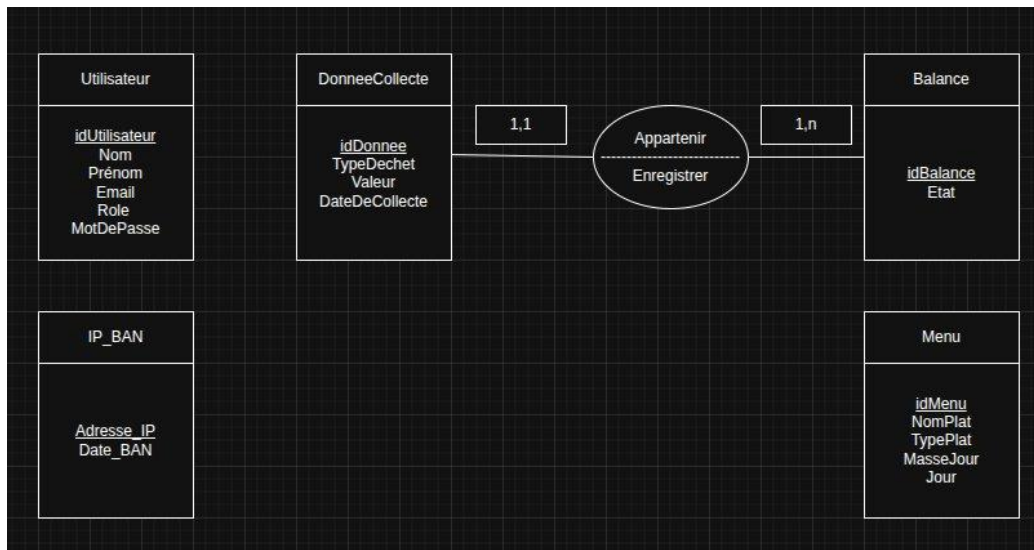
- Il dispose de bibliothèques MQTT fiables (paho-mqtt),
- Il permet un développement rapide et clair,
- Il est adapté aux scripts d'ingestion de données,
- Il facilite les tests grâce à des générateurs de données simples à écrire.

2.5 Conception détaillée

2.5.1 Développement logiciel

2.5.1.1 Tâche 2 : Configuration de la BDD

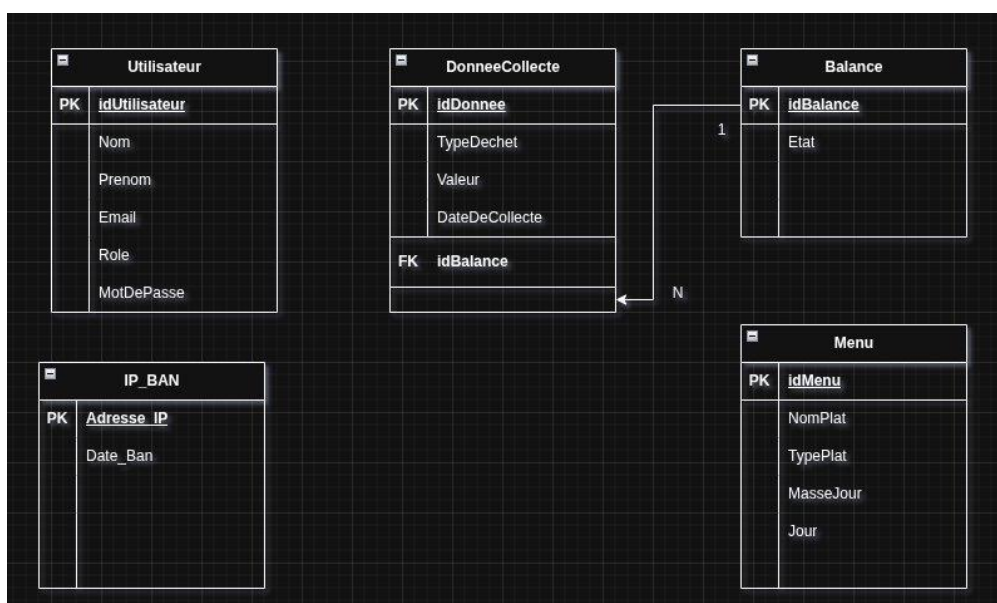
Schéma du MCD :



Le MCD ci-dessus permet de représenter les différentes entités du système ainsi que les relations fonctionnelles qui les relient, tout en intégrant les contraintes d'intégrité fonctionnelle nécessaires au bon fonctionnement de la base de données.

On peut voir que dans ma base de données, une Contrainte d'Intégrité Fonctionnelle (CIF) existe entre la table **Balance** et la table **DonneeCollecte**, car chaque donnée collectée doit obligatoirement être rattachée à une et une seule balance

Schéma relationnel de la base de données :



Explication des tables :

Table Utilisateur : Cette table stocke les comptes des personnes autorisées à accéder à l'application web. Elle contient les champs suivants : idUtilisateur (PK, entier auto-incrémenté), Nom (VARCHAR), Prénom (VARCHAR), Email (VARCHAR, unique), Role (VARCHAR — valeurs possibles : *agent*, *responsable*, *administrateur*), MotDePasse (VARCHAR, stocké sous forme hachée). Elle permet la gestion des droits d'accès conformément aux exigences RGPD.

Table Balance : Cette table enregistre les informations relatives à chaque balance connectée. Elle contient : idBalance (PK, entier auto-incrémenté | FK → DonneeCollecte.idBalance), Etat (TINYINT etat possible : 0 = éteint, 1 = allumé). Ainsi, la table Balance assure une identification claire des équipements et contribue à la qualité des données enregistrées dans la table **DonneeCollecte**, avec laquelle elle entretient une relation de type 1-N.

Table DonneeCollecte: Table centrale du système, elle stocke chaque pesée transmise via MQTT. Elle contient : idDonnée (PK, entier auto-incrémenté), TypeDechet (VARCHAR — Organique et Plastique), Valeur (FLOAT, en kg, plage valide : 0 à 500 kg), DateDeCollecte (DATETIME), idBalance (FK → Balance.idBalance). La relation est de type **1-N** : une balance peut générer plusieurs pesées.

Table Menu : Cette table permet d'associer les pesées aux menus servis. Elle contient : idMenu (PK), NomPlat (VARCHAR), TypePlat (VARCHAR), MasseJour (FLOAT), Jour (DATE). Elle est utilisée par l'application web (partie Étudiant 1) pour croiser les données de déchets avec les repas servis.

Table IP_BAN : Table de sécurité permettant de bloquer les adresses IP suspectes (tentatives de brute force, injection SQL, etc.). Elle contient : Adresse_IP (VARCHAR), Date_Ban (DATETIME). Elle contribue à la conformité RGPD et à la politique de sécurité du système.

2.5.1.2 Tâche 3 : Configuration du broker MQTT

Config 1 : Configuration de base

Fichier de configuration principal : /etc/mosquitto/conf.d/**broker.conf**

Contenu :

```
listener 1883
allow_anonymous true
```

Cette configuration permet d'accepter toutes les connexions sans authentification.

Tests de début :

mosquitto_pub -h ADRESSE_IP -t "NOM_DU_TOPIC" -m "CONTENUE_DU_MESSAGE"	Publication (En mode pub)
mosquitto_sub -h ADRESSE_IP -t "NOM_DU_TOPIC"	Abonnement (En mode sub)

Config 2 : Avec un utilisateur

Cette configuration ajoute une authentification par **nom d'utilisateur + mot de passe**.

Pour ajouter un utilisateur :

sudo apt install -y mosquitto mosquitto-clients	Etape 1 : Installer les outils de gestion des mots de passe
---	--

sudo mosquitto_passwd -c /etc/mosquitto/passwordfile	Etape 2 : Créer le fichier de mots de passe
sudo mosquitto_passwd /etc/mosquitto/passwordfile bal_test sudo mosquitto_passwd /etc/mosquitto/passwordfile bal_001	Etape 3 : Création des utilisateurs
Fichier de configuration principal : /etc/mosquitto/conf.d/broker.conf	Etape 4 : Configurer Mosquitto pour utiliser l'authentification
sudo systemctl restart mosquitto	Etape 5 : Redémarrer Mosquitto

Fichier de configuration principal : **/etc/mosquitto/conf.d/broker.conf**

Contenu :

listener 1883
allow_anonymous false # **Utilisateur anonyme non autorisé**
password_file /etc/mosquitto/passwd # **Fichier des mots de passe**

Fichier de configuration des utilisateurs : **/etc/mosquitto/passwd**

Contenu :

bal_001: "MDP HACHE" # **Balance**
bal_test: "MDP HACHE" # **Utilisateur Programme Test**

Tests de connexion :

mosquitto_sub -h ADRESSE_IP -t NOM_DU_TOPIC -u UTILISATEUR -P MOTDEPASSE	Abonnement (En mode sub)
mosquitto_pub -h ADRESSE_IP -t NOM_DU_TOPIC -m "CONTENU -u UTILISATEUR -P MOTDEPASSE	Publication (En mode pub)

Config 3 : Avec un utilisateur + TLS

Cette configuration ajoute une sécurité renforcée au broker Mosquitto grâce à :

- Une **autorité de certification interne (CA)**,
- Un **certificat serveur signé**,
- Le **chiffrement TLS**,
- L'**authentification utilisateur**,
- Des **ACL** pour contrôler les accès.

Création de l'autorité de certification (CA) :

La CA permet de signer les certificats utilisés par Mosquitto.

Dans le dossier : /etc/mosquitto/certs/ sudo openssl genrsa -out CA-Projet-Restauration-Scolaire.key 4096	Etape 1 : Création de la clé privée de la CA
sudo nano /etc/mosquitto/certs/openssl.cnf	Etape 2 : Création et modification du fichier de configuration de la CA
sudo openssl req -x509 -new -nodes \ -key CA-Projet-Restauration-Scolaire.key \ -sha256 -days 3650 \ -out CA-Projet-Restauration-Scolaire.crt \ -config openssl.cnf	Etape 3 : Génération du certificat CA auto-signé

Fichier de configuration de la CA : /etc/mosquitto/certs/openssl.cnf

GNU nano 6.2	openssl.cnf
<pre>[ca] default_ca = CA_default [CA_default] dir = /etc/mosquitto/certs database = \$dir/index.txt new_certs_dir = \$dir/newcerts certificate = \$dir/ca.crt serial = \$dir/serial private_key = \$dir/ca.key default_md = sha256 policy = policy_any email_in_dn = no rand_serial = no default_days = 3650 [policy_any] commonName = supplied [req] default_bits = 4096 prompt = no default_md = sha256 distinguished_name = dn [dn] CN = ProjetRestScolaire-CA</pre>	

Le fichier *openssl.cnf* présent dans /etc/mosquitto/certs/ définit la configuration de l'autorité de certification (CA) utilisée pour générer les certificats du broker Mosquitto. Il précise notamment

l'emplacement des fichiers de la PKI, l'algorithme de signature, la durée de validité des certificats ainsi que les informations du *distinguished name* de la CA.

Création du certificat serveur :

Ce certificat sera présenté par Mosquitto aux clients MQTT.

sudo openssl genrsa -out ServMosquitto.key 4096	Etape 1 : Générer la clé privée du serveur
sudo nano /etc/mosquitto/certs/server.cnf	Etape 2 : Création du fichier de configuration CSR (SAN)
sudo openssl req -new \ -key ServMosquitto.key \ -out ServMosquitto.csr \ -config server.cnf	Etape 3 : Générer la CSR du serveur
sudo openssl ca -config openssl.cnf \ -in ServMosquitto.csr \ -out ServMosquitto.crt \ -extensions server_cert	Etape 4 : Signer le certificat serveur avec la CA

Fichier de configuration CSR: /etc/mosquitto/certs/server.conf

```
[ req ]
default_bits    = 2048
prompt         = no
default_md      = sha256
req_extensions  = req_ext
distinguished_name = dn

[ dn ]
C = FR
ST = Bretagne
L = Brest
O = Projet-Restauration-Scolaire
OU = Mosquitto
CN = « IP_Proxmox »

[ req_ext ]
subjectAltName = @alt_names

[ altnames ]
IP.1 = « IP_Proxmox »
```

Le fichier *server.conf* situé dans **/etc/mosquitto/certs** définit les paramètres nécessaires à la génération du CSR (Certificate Signing Request) du serveur Mosquitto, notamment les informations d'identité du serveur, l'algorithme de signature, la taille de clé ainsi que les extensions TLS comme le *Subject Alternative Name*.

Dossier contenant les certificats : **/etc/mosquitto/certs**

```
CA-Projet-Restauration-Scolaire.crt : Certificat CA
CA-Projet-Restauration-Scolaire.key : Clé privée CA
ServMosquitto.crt : Certificat serveur signé
ServMosquitto.csr : CSR du serveur
ServMosquitto.key : Clé privée serveur
Openssl.cnf : Configuration de la CA
Server.cnf : Configuration CSR serveur (SAN)
```

Fichier de configuration principal : **/etc/mosquitto/conf.d/broker.conf**

```
listener 8883
protocol mqtt
password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/acl
cafile /etc/mosquitto/certs/CA-Projet-Restauration-Scolaire.crt
certfile /etc/mosquitto/certs/ServMosquitto.crt
keyfile /etc/mosquitto/certs/ServMosquitto.key

allow_anonymous false
```

Le fichier *broker.conf*, situé dans **/etc/mosquitto/conf.d** définit la configuration principale du broker Mosquitto : il précise le port sécurisé utilisé, les fichiers d'authentification, les règles d'accès ainsi que les certificats TLS nécessaires au chiffrement des communications.

Fichier ACL : **/etc/mosquitto/acl**

```
user bal_001
topic write balance/bal_001/data -> topic principal
topic write balance/bal_001/data-2 -> topic de secours
topic read balance/bal_001/# -> lecture sur tous les topics

user bal_test
topic write balance/bal_test/data -> topic principal
topic write balance/bal_test/data-2 -> topic de secours
topic read balance/bal_test/# -> lecture sur tous les topics
```

Tests de connexion :

mosquitto_sub -h ADRESSE_IP -p 8883 -u "UTILISATEUR" -P "MOTDEPASSE" --cafile CHEMIN_DU_CERTIFICAT_CA -t "NOM_DU_TOPIC"	Abonnement (En mode sub)
mosquitto_pub -h ADRESSE_IP -p 8883 -u "UTILISATEUR" -P "MOTDEPASSE" --cafile CHEMIN_DU_CERTIFICAT_CA -t "NOM_DU_TOPIC" -m "CONTENU_DU_MESSAGE"	Publication (En mode pub)

2.5.2 Configuration/Installation réseau ou de services

2.5.2.1 Tache 1 : Installation sur le conteneur Proxmox des services

Procédure installation Nodejs:

Dans un terminal connecté au conteneur Proxmox:

sudo apt update && sudo apt upgrade -y	Étape 1 : Mettre à jour le système
curl -fsSL https://deb.nodesource.com/setup_lts.x sudo -E bash -	Étape 2 : Ajouter le dépôt NodeSource
sudo apt install -y nodejs	Étape 3 : Installer Node.js
node -v npm -v	Étape 4 : Vérifier l'installation

Procédure installation MariaDB:

Dans un terminal connecté au conteneur Proxmox:

sudo apt update && sudo apt upgrade -y	Étape 1 : Mettre à jour le système
apt install mariadb-server -y	Étape 2 : Installer le paquet
systemctl status mariadb	Étape 3 : Vérifier que le service tourne
mysql_secure_installation	Étape 4 : Sécuriser l'installation

Action à mettre dans **mysql_secure_installation** :

- Définir un mot de passe root
- Désactiver l'accès root distant
- Supprimer les bases de test
- Recharger les privilèges

Procédure Installation Mosquitto MQTT :

Dans un terminal connecté au conteneur Proxmox:

sudo apt install mosquitto mosquitto-clients -y	Étape 1 : Installe le broker Mosquitto + outils clients
sudo systemctl start mosquitto	Étape 2 : Démarre le service Mosquitto
sudo systemctl enable mosquitto	Étape 3 : Active le démarrage automatique au boot
sudo systemctl status mosquitto	Étape 4 : Vérifie que le service fonctionne

Tests de début :

mosquitto_pub -h ADRESSE_IP -t "NOM_DU_TOPIC" -m "CONTENUE_DU_MESSAGE"	Publication (En mode pub)
mosquitto_sub -h ADRESSE_IP -t "NOM_DU_TOPIC"	Abonnement (En mode sub)

2.5.2.1 Tache 4 : Réception des données sur le broker et envoi vers la BDD

Description des données reçues

Les données envoyées par les balances connectées transitent via le broker MQTT sous forme de **messages JSON**. Chaque message représente une pesée unique et contient les informations nécessaires à l'insertion dans la table DonneeCollecte.

Exemple de données :

```
{"Valeur" : 12.0, "TypeDechet" : "Dechet organique", "Date" : "2026-04-01 13 :53 :45", "idBalance" : "1", "Topic choisi" : "data/Donnee"}
```

Chaque message possède les champs suivants :

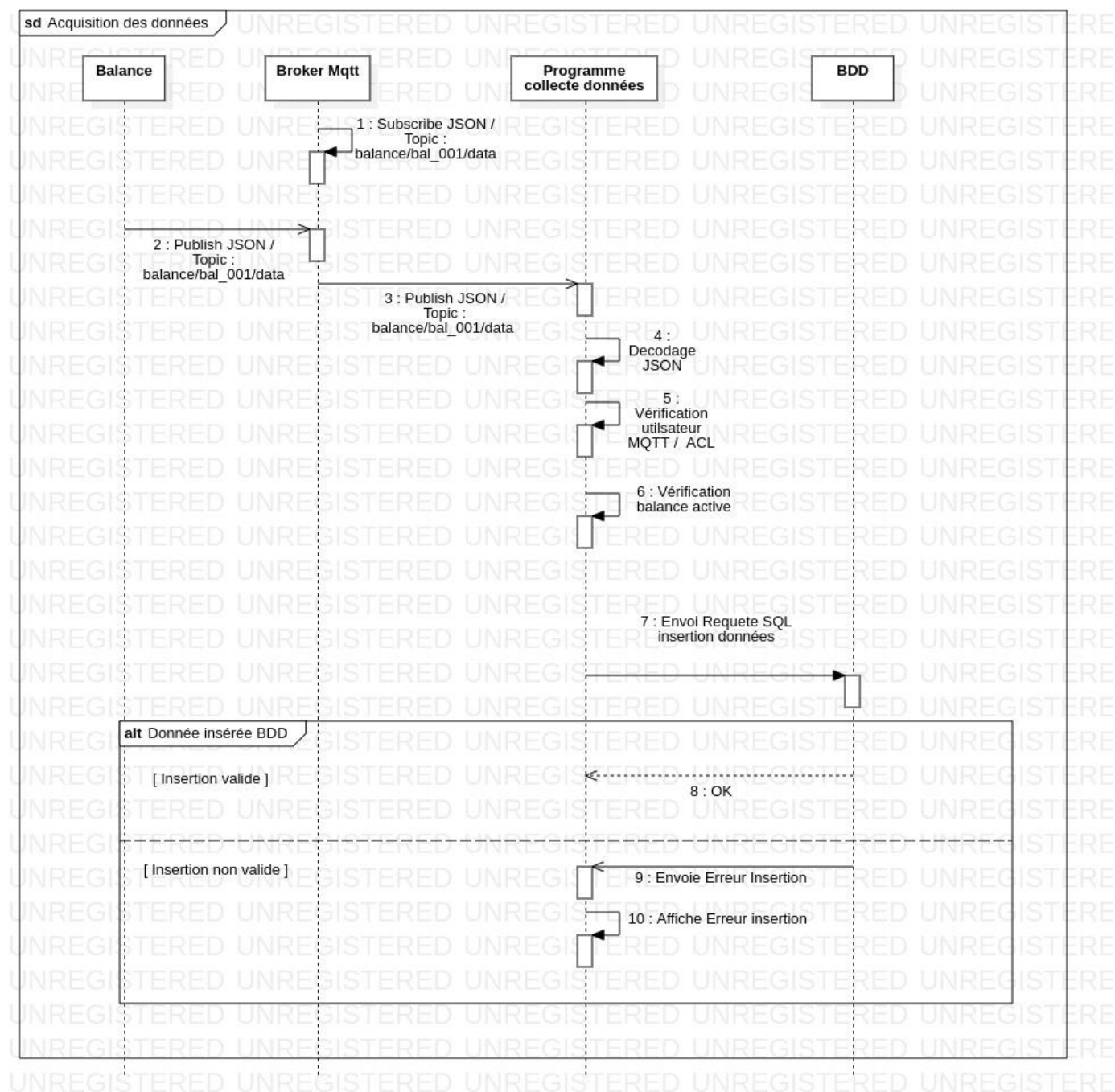
- **Valeur** : Masse mesuré (FLOAT) # La Masse sera stocké sous la forme de kg dans la BDD.
- **TypeDechet** : type de déchet (VARCHAR)
- **Date** : date et heure de la pesée (DATETIME)
- **idBalance** : identifiant de la balance (INT)
- **Topic choisi** : topic MQTT utilisé pour l'envoi (STRING)

Processus de traitement :

Le script Python est **abonné au topic MQTT** correspondant.

A chaque message reçu :

- Le JSON est **décodé** et **validé** (types, champs obligatoires, plage de valeurs).
- Les données sont **normalisées** (ex : type de déchet).
- Une requête SQL **INSERT** est exécutée vers la table DonneeCollecte.

Diagramme de séquence :

Ce diagramme de séquence permet de représenter le cas d'utilisation **Acquisition des données**, en illustrant les différentes interactions entre la balance, le broker MQTT, le programme de collecte et la base de données lors de l'envoi et du traitement d'une mesure.

Extrait de code :Réception MQTT et insertion en base de données

```
def on_message(client, userdata, msg) :
    print("\n--- Message reçu ---")
    print("Topic :", msg.topic)
    print("Payload:", msg.payload.decode())

    try:
        data = json.loads(msg.payload.decode())
    except:
        print("JSON non valide.")
    return

    parts = msg.topic.split("/") # Decomposition du topic
    if len(parts) < 3 : # Verification de la composition du topic
        print("Topic invalide.")
    return

    mqtt_username = parts [1] # partie 1 : utilisateur
    action = parts [2] # partie 2 : utilisateur

    # Verification des ACL : permission
    allowed_topics = ACL.get(mqtt_username, {}).get("write", [])
    if msg.topic not in allowed_topics:
        print("TOPIC NON AUTORISÉ pour", mqtt_username, " :", msg.topic)
    return

    # Balance Test
    idBalance = ID_BALANCE
    etat = 1

    if etat == 0 :
        print("Balance inactive :", idBalance)
    return

    # Insertion des données
    if action == "data" :
        try:
            curseur.execute(
                """
                INSERT INTO DonneeCollecte (Valeur, TypeDechet, DateDeCollecte, idBalance)
                VALUES (%s, %s, %s, %s)
                """,
                (data["Valeur"], data["TypeDechet"], datetime.now(), idBalance))
            BDD.commit()
            print("Donnée insérée pour la balance", idBalance)
        except Exception as e :
            print("Erreur insertion DonneeCollecte :", e)
        else:
            print("Action non gérée :", action)
```

Explication du fonctionnement :**1. Réception du message MQTT**

Dès qu'un message arrive sur un topic auquel le programme est abonné, la fonction `on_message()` est automatiquement déclenchée. Le topic et le contenu du message sont affichés pour permettre un suivi en temps réel.

2. Décodage du JSON

Le programme tente ensuite de convertir le payload reçu en dictionnaire Python via `json.loads()`. Si le JSON est invalide ou mal formé, le message est ignoré afin d'éviter toute insertion incorrecte en base.

3. Analyse du topic et identification de l'émetteur

Le topic est découpé en plusieurs parties pour extraire :

- Le nom d'utilisateur MQTT,
- L'action demandée (ex : data). Cette structure permet de vérifier que le message respecte bien le format attendu.

4. Vérification des permissions (ACL)

Le programme contrôle ensuite si l'utilisateur MQTT a le droit d'écrire sur ce topic. Si le topic n'est pas autorisé pour cet utilisateur, la donnée est rejetée pour garantir la sécurité du système.

5. Vérification de l'état de la balance

Avant toute insertion, le programme vérifie que la balance associée est active. Une balance inactive ne peut pas envoyer de données, ce qui évite d'enregistrer des mesures provenant d'un équipement hors service.

6. Insertion en base de données

Si l'action correspond à data, le programme insère la mesure dans la table **DonneeCollecte** avec :

- La valeur mesurée,
- Le type de déchet,
- La date de réception,
- L'identifiant de la balance. En cas de succès, un message confirme l'insertion ; sinon, une erreur détaillée est affichée.

2.6 Tests unitaires

Test n° 1 : Envoie de données - Balance vers broker

Élément testé :	Programme Test en mode subscribe / Balance Connecté			
Objectif du test :	Vérifié que les données envoyées transitent bien sur le broker			
Nom du testeur :	Aballea Killian / Younn D		Date :	06/05/2026
Moyens mis en œuvre :	Logiciel : Arduino, Visual Studio Code, Terminal Linux	Matériel : Balance Connecté, PC	Outil de développement : X	
Procédure du test :				
Id	Description du vecteur de test	Résultat attendu	Résultat obtenu	Validation (O/N)
1	Insertion en continu	Donnée attendue	492	O
2	Insertion en continu	Donnée attendue	492	O
3	Insertion en continu	Donnée attendue	490	O
Conclusion du test :	L'ajustement de la masse en kilogrammes a été réalisé juste après la mesure. Le poids obtenu correspond à une bouteille d'eau d'un litre remplie à moitié, ce qui confirme la cohérence de la valeur et valide le test.			

Test n° 2 : Reception des données en envoi vers la BDD – Script réception données

Élément testé :		Script envoi de données / Programme Test		
Objectif du test :		Vérifié si les données envoyées sont bien traitées et envoyées dans la BDD correctement		
Nom du testeur :		Aballea Killian	Date :	06/05/2026
Moyens mis en œuvre :		Logiciel : Visual Studio Code, Terminal Linux	Matériel : PC	Outil de développement : X
Procédure du test :				
Id	Description du vecteur de test	Résultat attendu	Résultat obtenu	Validation (O/N)
1	Traitement des données via le Script	Insertion dans la BDD fonctionnelle	Voir image tests	O
2	Traitement des données via le Code	Insertion dans la BDD fonctionnelle	--- Message reçu --- Topic : balance/bal_test/data Payload: {"Valeur" : 2.0, "TypeDechet" : "Dechet organique", "Date" : "2026-05-11 15 :00 :00"} Donnée insérée pour la balance 1	O

Conclusion du test :	L'envoi dans la BDD de 2 manières a bien fonctionné donc les tests sont concluants
----------------------	--

Résultat table DonneeCollecte :

95	organique	5.312	2026-05-07 14:16:37	1
----	-----------	-------	---------------------	---

2.7 Problèmes rencontrés

Problèmes rencontrés	Solutions trouvées
TLS avec le Broker : Problème avec le certificat : non auto signée Problème de lecture avec les certificats	Récréer une nouvelle autorité de certification afin de générer un nouveau certificat. Redonner les bons droits aux certificats pour qu'il soit lisible depuis l'extérieur.
Ancien script plus fonctionnel avec la version actuelle du projet.	Refaire le script en lui mettant la bonne version en programme et en le transformant en service afin qu'il tourne en continu.
Confusion avec le programme et la Balance au niveau de l'ID.	Changement dans le programme en passant l'IP de la balance en 2 dans le programme.

3. Bilan de la réalisation personnelle

3.1 Points du projet validés

J'ai pu mener à bien l'ensemble des tâches qui m'étaient attribuées, notamment la **collecte et le stockage des données**, ainsi que l'installation du serveur et le déploiement des services nécessaires, incluant le serveur web et la base de données. J'ai également **défini la base de données**, son **MCD** et son **schéma relationnel**, ce qui a permis de structurer efficacement les informations du système. La **sécurisation des accès** a été réalisée à travers la configuration des utilisateurs, des ACL, des certificats TLS et du broker MQTT. J'ai aussi établi une **liste de tests d'intrusion essentiels** afin d'identifier les principales menaces auxquelles le système doit résister bien que certains n'ont pas été testés. Enfin, j'ai **configuré le broker Mosquitto** et généré les **requêtes SQL d'interrogation** nécessaires à l'exploitation des données collectées.

3.2 Parties à développer

La **vérification de la conformité au RGPD** a été amorcée, notamment par l'analyse des données collectées et des besoins de conservation, mais cette partie nécessite encore un approfondissement pour être totalement finalisée. La **gestion des logs**, bien qu'identifiée comme un élément important pour le suivi et la détection d'erreurs, n'a pas encore été mise en place.

3.3 Améliorations possibles

Une amélioration pertinente serait la création d'un **script de supervision continue** permettant de vérifier en temps réel l'état de chaque balance afin de détecter rapidement les anomalies ou les déconnexions. La mise en place d'un **système de gestion des logs plus complet** permettrait également d'améliorer le diagnostic des erreurs et d'assurer un suivi plus précis du fonctionnement global du système.

3.4 Conclusion personnelle

Ce projet m'a permis de perfectionner mes compétences en réseaux, en développement et en cybersécurité, tout en travaillant sur une architecture IoT complète allant du broker MQTT jusqu'à la base de données. Mettre en place l'infrastructure, configurer le chiffrement TLS, concevoir la base de données et développer le script Python m'a permis de comprendre concrètement le fonctionnement d'un système fiable et sécurisé. Les difficultés rencontrées, notamment celles liées aux certificats et à la cohérence des identifiants, m'ont appris à diagnostiquer efficacement de vrais problèmes techniques et à renforcer ma rigueur.

Au-delà des aspects techniques, ce projet a également été une véritable expérience de **travail d'équipe**. La coordination avec les autres membres du groupe, le partage des tâches, les échanges réguliers et la résolution collective de certains blocages ont joué un rôle essentiel dans l'avancement du projet. Cette collaboration m'a permis d'améliorer ma communication, de mieux structurer mon travail et de contribuer à une dynamique de groupe efficace.

Enfin, ce projet a confirmé mon intérêt pour un domaine mêlant développement, automatisation et intelligence artificielle. L'année prochaine, en intégrant l'ISEN en FISA, mon objectif est de m'orienter vers des technologies plus avancées, notamment l'IA, l'analyse de données et les architectures logicielles modernes. Les compétences acquises ici constituent une base solide pour progresser dans ces domaines et continuer à développer des systèmes intelligents, performants et sécurisés.